QCon
全球软件开发大会

InfoQ 技术大会

ProphetKV：让 KV Cache 复用突破前缀匹配

由用户问题驱动的选择性重算

ICML 2026 poster

彭德跃

北京焱融科技 首席架构师



一条主线：把计算留给问题关心的内容

关于这项工作 本次分享基于 ProphetKV（ICML 2026 poster），由哈尔滨工业大学（深圳）与北京焱融科技合作完成；我代表焱融团队介绍这项工作，以及它在我们存储体系中的落地思考。

1. **复用文档块 KV**
   文档顺序变了也能用
2. **用查询注意力挑 token**
   挑出问题真正需要的部分
3. **只重算被选中的 token**
   20% 预算恢复块间注意力

精度由「选谁」决定；时延由「算多少 + 读多少」决定

20%	⩾95%	4.6×
重算比例	精度保留，基本与全量持平	prefill 加速（16K，对比全量重算）

数据：Wang et al., ProphetKV, ICML 2026 poster；精度保留为 Tab.1 各平均值相对全量的比例（95.4%–101.2%），加速比为 Llama-3.1-8B / H100 / 16K 实测

QCon
全球软件开发大会

InfoQ 技术大会

目录

01 上下文一拼装，前缀复用就失效

02 选择性重算：已有方案与瓶颈

03 ProphetKV（一）：用问题决定重算谁

04 ProphetKV（二）：两阶段执行

05 实验结果

06 存储视角：跨块复用之后的成本账

01

上下文一拼装，前缀复用就失效

从前缀匹配到片段级 KV

前缀复用为何在上下文拼装时失效

前缀复用为什么无损

- 因果注意力：第 i 个 token 的 K、V 只依赖第 1…i 个 token
- 前 m 个 token 相同 ⇒ 每一层的 KV 都相同 ⇒ 直接复用，输出不变

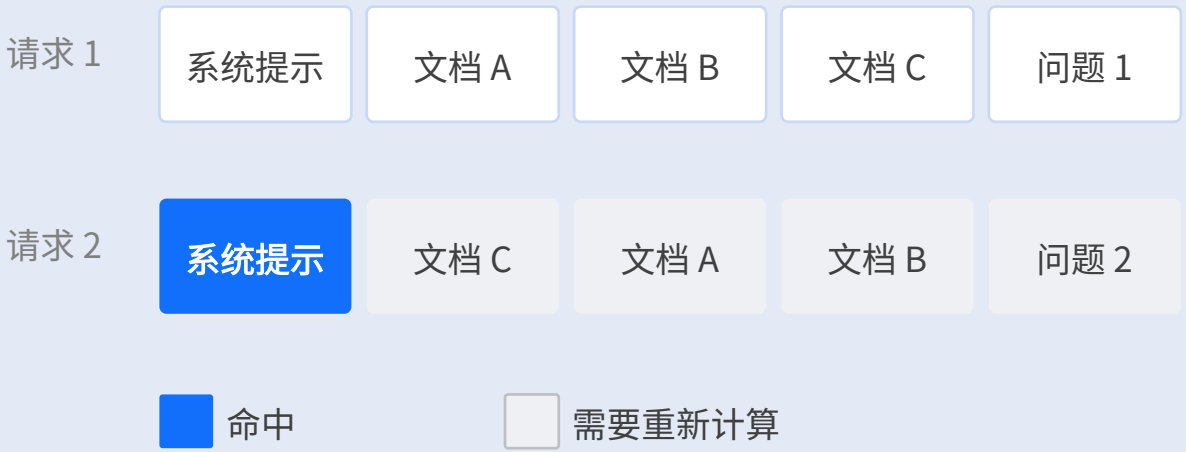
可复用长度 = LCP(已缓存序列, 新请求)

LCP：最长公共前缀；vLLM 按块、SGLang 按 token 查找

为什么拼装式输入用不上

- 片段的组合和顺序随任务变化：检索按相关度排序、技能按需加载……
- 前缀要求「内容和顺序都一致」，第一个不同的块之后全部作废

以检索为例：同一批文档，换个问题



结果

文档 A、B、C 的 KV 都已经在缓存里，却只有系统提示命中

推算：同样 6 个块、顺序随机时，首块相同的概率 1/6，全部同序 1/720（假设条件，非实测）

看不见的账：为同一段文字反复付费

一个示例场景：企业知识库客服，每天约 100 万次提问；每次检索 6 个文档块，约 6000 个 token 来自高频文档（产品手册、政策条款），这些内容每天被反复检索

GPU 账

≈ 83

GPU 小时 / 天

100 万 × 6000 token = 60 亿 token 被重复 prefill；按单卡 prefill 吞吐 2 万 token/s 估算

等待账

0.5–1.5 s

每个请求多等的时间

论文实测 4K–8K 上下文的全量 prefill 耗时（HuggingFace 实现，生产引擎会更低）；直接影响体验

投资账

≈ 0

文档部分的前缀命中率

只有系统提示命中；块 KV 已经算好、存好，却因为顺序不同命不中，缓存容量和带宽的投入基本沉没

问题不在于「有没有缓存」，而在于前缀式的缓存组织和拼装式的输入不匹配

示例场景与请求量、吞吐均为假设，用于说明量级；实际请以自家业务数据替换

不只是 RAG：越来越多的负载在「拼装」上下文

负载	拼装的片段	组合 / 顺序会变吗	ProphetKV	其它工作的验证
文档问答（RAG）	检索出的文档块	随问题变化	**论文实验覆盖**	CacheBlend、QCFuse 等
Agent 技能・工具・记忆	按任务加载的 skill、工具说明、记忆	随任务变化	**未验证**	Attuner、ReCache（需训练）
多智能体协作	其它 agent 的输出报告	以不同组合引用	**未验证**	KVCOMM、KVShareArena
上下文压缩之后	保留下来的历史片段	删掉一段，后面错位	**未验证**	暂无
编码 agent	同一批源文件	以不同顺序重读	**未验证**	部分（KVCOMM 协作编码）

共同点

复用单位从「会话前缀」变成「上下文片段」：片段可复用，但出现的位置和组合每次不同

口径

ProphetKV 实验全是问答式长上下文任务，其它负载待验证；SGLang 社区已有位置无关复用的 RFC 与原型（截至 2026-10 未合并）

参考：Attuner（2026）；ReCache（2026）；KVCOMM（NeurIPS'25）；KVShareArena（2026）；SGLang RFC #30928

三种做法的取舍：快和准，前缀复用只能在开头兼得

全量重算

每次对全部文档重新 prefill

✓ 准确　✗ 快

跨块注意力完整；时延随上下文超线性增长

前缀复用

只复用开头完全相同的部分

✓ 准确　✗ 时延

无损；但片段一换顺序，通常只有第一个块命中，只省第一块，时延仍高

直接拼接

各块单独预计算 KV，按新顺序拼起来

✓ 快　✗ 准确

命中率高，但块与块之间的注意力全部缺失

能不能既快又准？ 直接拼接 + 只重算少量 token，恢复关键的块间注意力 → 选择性重算（第 02 部分）

改绘自 来源：Wang et al., ProphetKV, ICML 2026 poster, Fig. 1

Prompt Cache：非前缀复用的起点

做法：把可复用的片段定义成「模块」，单独预计算



运行时按需拼接，顺序可以不同；只有问题现算

要点

- 用 schema 声明可复用的模块：系统提示、模板、上下文文档
- schema 预留位置，保证拼接后位置编码正确
- 论文报告：长 prompt 下 TTFT 在 GPU 上降约 8 倍

贡献

第一次让「不在开头」的内容也能复用 KV，打破了严格的前缀限制

缺口

模块之间互相看不见：跨模块的注意力完全缺失。

模板、系统提示这类关联弱的场景够用；需要跨片段推理（如多文档问答）就会出错

来源：Gim et al., Prompt Cache: Modular Attention Reuse for Low-Latency Inference, MLSys 2024；Yao et al., CacheBlend, EuroSys 2025

直接拼接丢了什么：跨块推理需要同时命中两条事实

块 1	块 2	块 3	问题
…… John 的房子在伦敦 ……	…… Alice 的房子在巴黎 ……	…… 周一 Alice 住在 John 家 ……	**周一 Alice 住在哪个城市？**

直接拼接

块 3 预计算时没见过块 1

「John 家」没有和「伦敦」建立联系，模型只能借助块 2 的「Alice 的房子在巴黎」作答

✗ 巴黎

全量 prefill

块 3 能看到块 1

「周一住在 John 家」+「John 的房子在伦敦」⇒ 伦敦

✓ 伦敦

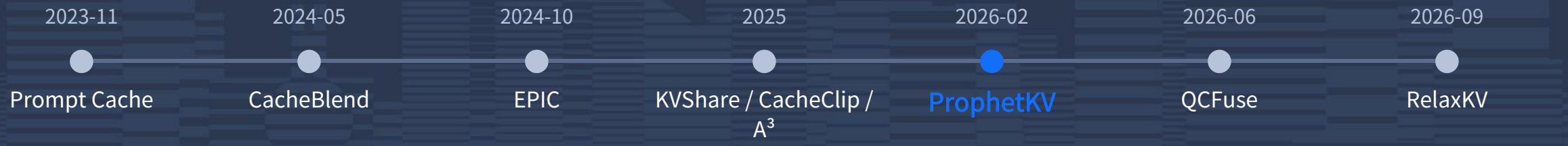
选择性重算要修的，正是这类「需要跨块关联」的 token

改绘自 来源：Wang et al., ProphetKV, ICML 2026 poster, Fig. 2

02

选择性重算：已有方案与瓶颈

有限的重算预算，该给谁



完整时间线见附录 B1

CacheBlend：选择性重算的起点

1

第一层找偏差

对所有 token 算预计算 KV 与实际 KV 的偏差，选最大的一批

2

逐层收缩

后续各层只在上一层候选里重算并继续筛选；默认约 15%

3

与加载流水

第 i 层重算与第 i+1 层 KV 读取并行，重算被读取掩盖

局限：选择依据（KV 偏差）与当前问题无关；用浅层信息决定深层重算谁

存储视角的亮点

论文示例：Llama-7B、4K 上下文、重算 15%

每层重算 3 ms，从 NVMe SSD 读一层 16 ms

→ KV 可以放在更便宜的 SSD 上，而不增加时延

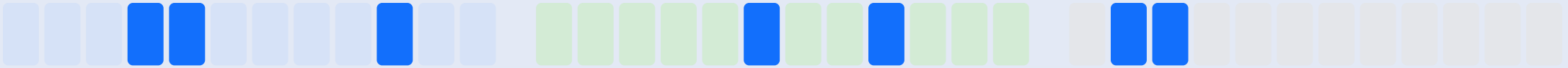
效果：相比全量重算，TTFT 降 2.2–3.3 倍，吞吐提升 2.8–5 倍

影响：EuroSys 2025 最佳论文；在 LMCache 中实现（旧版已停止维护，新架构接入仍在开发，截至 2026-10）

ProphetKV 保留「选择性重算」的框架，换掉的是「选谁」的依据

来源：Yao et al., CacheBlend, EuroSys 2025；LMCache 文档与 GitHub issue

选择性重算：大部分复用，少量重算，关键是挑谁



示意：三个文档块拼接，约 20% 的 token（蓝色）用完整上下文重新计算；计算量约为全量的 20%

现有方法的四类选择依据

按位置

EPIC（ICML'25）

每个块开头的若干 token

开销最低；与输入无关

按 KV 偏差

CacheBlend（EuroSys'25）

浅层算偏差，选偏差大的，逐层收缩

能与 KV 加载流水；用浅层信息判断深层

按偏差加权

KVShare（arXiv 2025）

选加权偏差最大的 token

动态选择；同样不看问题

辅助模型的查询注意力

CacheClip（arXiv 2025）

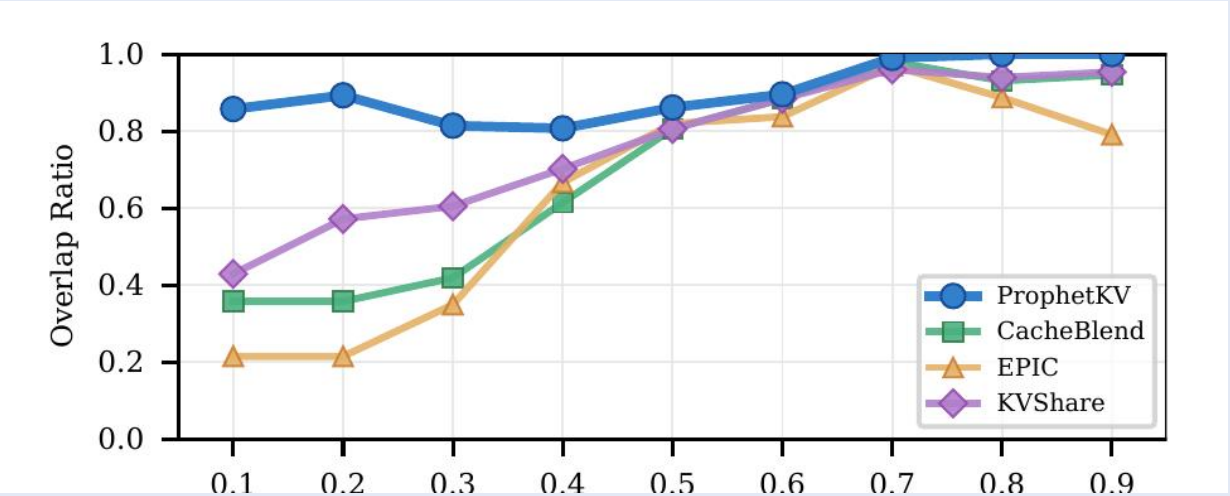
用小模型估计问题关注哪里

开始利用问题；需要额外模型

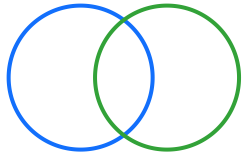
前三类的目标是把全局的块间注意力修好：修好的 KV 与问题无关、可以跨问题复用；但预算有限时，修的不一定是当前问题需要的部分

详见附录 B1

挤出效应：有限的预算被「看起来重要」的 token 占满



本图参照集合 = 问题真正关注的 token；横轴为重算比例 p



重合率 = |选中 ∩ 参照| ÷ |参照|

1 = 完全一致，0 = 毫不相干；随机挑 p 比例的 token，重合率约为 p

低预算下差距最大

- 重算比例 0.1–0.3 时，与问题无关的选择方法，重合率多数低于 0.5
- 按问题注意力选择，同样预算下保持在 0.8 以上

反映到精度（RULER MultiValue，20% 预算）

按 KV 偏差选择，相对全量损失 27%–42%

全局显著 ≠ 当前问题相关：预算花在了全局看起来重要的 token 上，当前问题真正需要的反而没排上

来源：Wang et al., ProphetKV, ICML 2026 poster, Fig. 3、Tab. 1

03

ProphetKV（一）：用问题的注意力，决定重算谁

在 decode 之前，利用用户问题的注意力

问题在末尾，是一个免费的先验



prefill 阶段就能拿到：问题的每个 token 对各文档 token 的注意力

固定结构

多数问答类模板是「上下文在前，问题在后」

看得最全

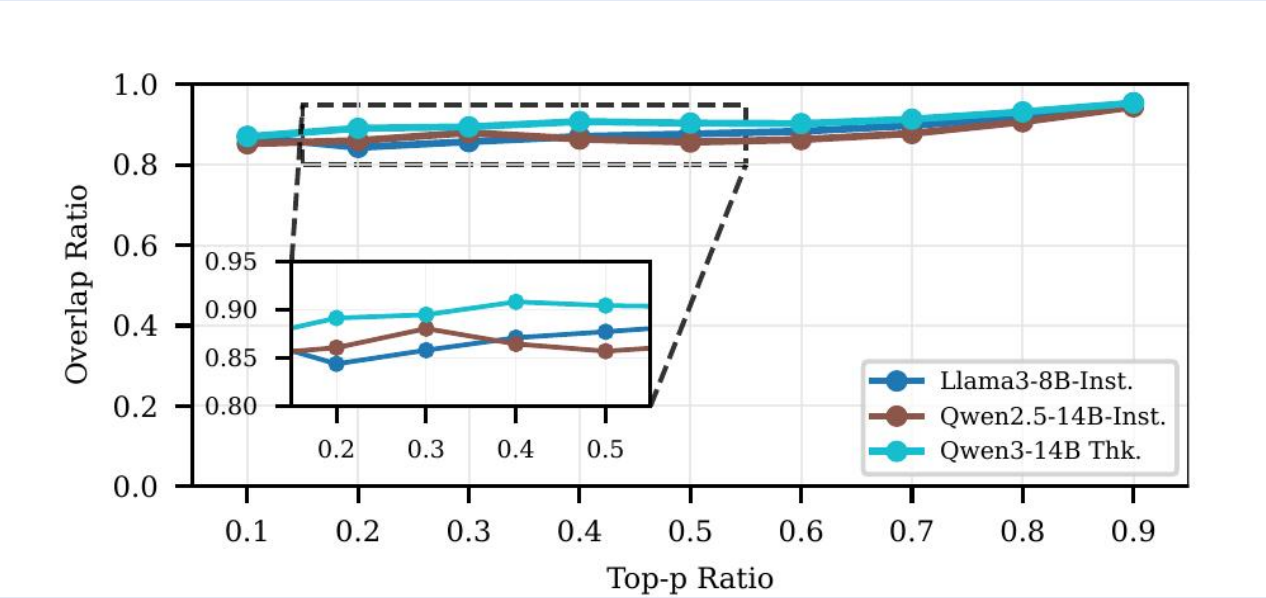
因果掩码下，只有末尾的问题 token 能看到全部上下文

不用等

decode 还没开始，问题已经「指出」答案可能在哪里

问题不在末尾时这个先验会变弱，见第 06 部分「三个代价」

问题的注意力，预示了生成时的关注点



本图参照集合 = 生成 token 关注的 token；比较对象 = 问题 token 关注的 token（各取 top-p）

观测

三个模型（Llama3-8B、Qwen2.5-14B、Qwen3-14B 推理模式）在各个比例下，重合率都在 0.85 以上

注意

这是经验信号，不是精度保证；长推理、长生成时关注点可能漂移

推论：用问题的注意力挑 token，挑中的正是生成时会用到的

来源：Wang et al., ProphetKV, ICML 2026 poster, Fig. 4

误差从哪来：问题视角下的语义损失

① 用问题代替生成 token

$$\Phi(n, t) \approx \Phi(t)$$

Φ(t)：问题各 token 对 token t 的平均注意力

② 问题读到的内容，误差有多大

$$L = \| \Sigma \Phi(t) \cdot V(t) - \Sigma \Phi'(t) \cdot V'(t) \|$$
$$\leqslant \Sigma \| \Phi(t) \cdot V(t) - \Phi'(t) \cdot V'(t) \|$$

拆成逐 token 之和，每个 token 可以单独打分

③ 单个 token 的误差

$$\Phi(t) \cdot V(t) - \Phi'(t) \cdot V'(t) = \Delta\Phi(t) \cdot V'(t) + \Phi(t) \cdot \Delta V(t)$$

带 ' 的是复用 KV 上的近似值，不带 ' 的是全量 prefill 的真值

$\Delta\Phi(t) \cdot V'(t)$

看错了地方

缺少跨块注意力，问题对这个 token 的关注度偏了

$\Phi(t) \cdot \Delta V(t)$

看到的内容不准

这个 token 自己的 V 没有融合其它块的信息

目标：用有限的预算，消掉误差最大的那些 token

简化自 来源：Wang et al., ProphetKV, ICML 2026 poster, Sec. 4.2

选哪个量来打分：要有效，还要在重算前就能看到

|ΔΦ(t)|

注意力偏差

效果最好

要先有真值，重算前拿不到

Φ′(t)

复用 KV 上的问题注意力

效果接近最好

只跑问题的几个 token 就能拿到

‖ ΔV(t) ‖

V 的偏差（≈ CacheBlend 思路）

效果接近

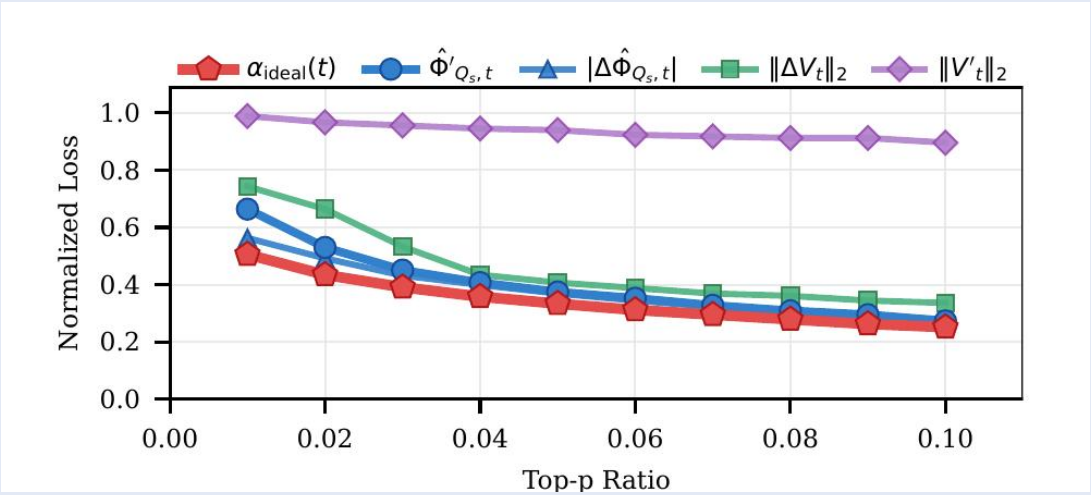
要先有真值，重算前拿不到

‖ V′(t) ‖

V 的大小

效果差

可观测，但没用



来源：Wang et al., ProphetKV, ICML 2026 poster, Fig. 6

ProphetKV 的选择

打分 α(t) = Φ′(t)

左图：按各因子选 token 后剩下的误差，越低越好。Φ′ 紧贴理想曲线，又不需要全量 prefill

选择几乎不花钱：只算「问题那几行」

完整注意力

$N \times N$

问题打分

$|Q| \times N$

|Q|（问题长度）远小于 N（上下文长度），选择开销随上下文线性增长，全量 prefill 是平方增长

上下文	Stage I	Stage II	全量 prefill	占比
4K	0.015 s	0.138 s	0.464 s	3.2%
8K	0.036 s	0.319 s	1.478 s	2.4%
16K	0.050 s	1.082 s	5.231 s	1.0%

Llama-3.1-8B，H100；Stage I 为问题打分，Stage II 为部分重算

≈1%

16K 上下文时，选 token 的开销只占全量 prefill 的 1%

时间主要花在 Stage II 的部分重算上；上下文越长，选择开销占比越低

数据：ProphetKV 作者在 OpenReview 审稿回复中补充的时延拆分（ICML 2026 poster）

能力边界：问题要比上下文短得多

Stage I 开销 ≈ |Q| / N 倍的全量 prefill（逐层只算问题那几个 token 的前向）；Stage II 本身就要约 20% 的计算量

场景	问题长度 |Q|	上下文 N	Stage I 占比（估算）
典型文档问答（RAG）	20–100	4K–32K	0.1%–2.5%
问题里带多轮对话历史	500–2K	8K	6%–25%
问题和上下文差不多长	≈ N	—	接近再做一次 prefill

实测点

- 4K：3.2%
- 8K：2.4%
- 16K：1.0%

Llama-3.1-8B，短问题；与估算量级一致

边界一：开销

|Q| 明显小于 Stage II 预算（约 20% 的 N）时，选择不是瓶颈

边界二：信号

问题越长、越杂，平均注意力越分散，挑 token 的准确度会下降

表中占比为按 |Q| / N 的估算，非实测；实测点来自 ProphetKV 作者 OpenReview 回复

04

ProphetKV（二）：两阶段重算，在多层模型里高效执行

先跨层选 token，再连续更新 KV

难点：层间依赖的死结



死结

每层真正需要重算的 token 不同；想知道深层该选谁，得先把前面所有层算完——这正是想省掉的开销

捷径一：只看第一层（CacheBlend 思路）

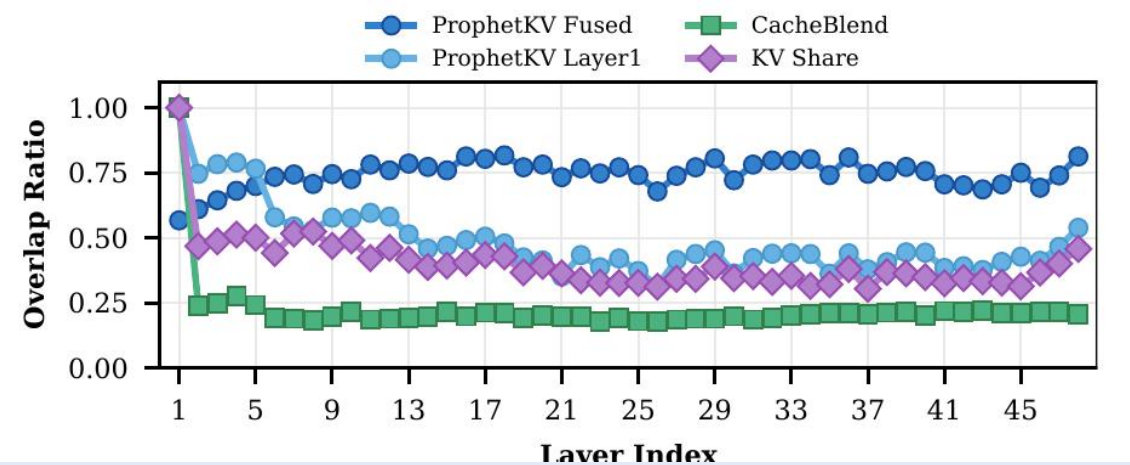
便宜，但浅层信号代表不了深层

捷径二：逐层边选边算

每层停下来等决策，无法连续执行

ProphetKV 的出路：问题 token 的前向本身就是逐层的，不需要先重算上下文，就能拿到每一层的打分

只看第一层不够：跨层信息才可靠



本图参照集合 = 每层真正该重算的 token；比较对象 = 各方法所选 token；Fused 为跨层融合

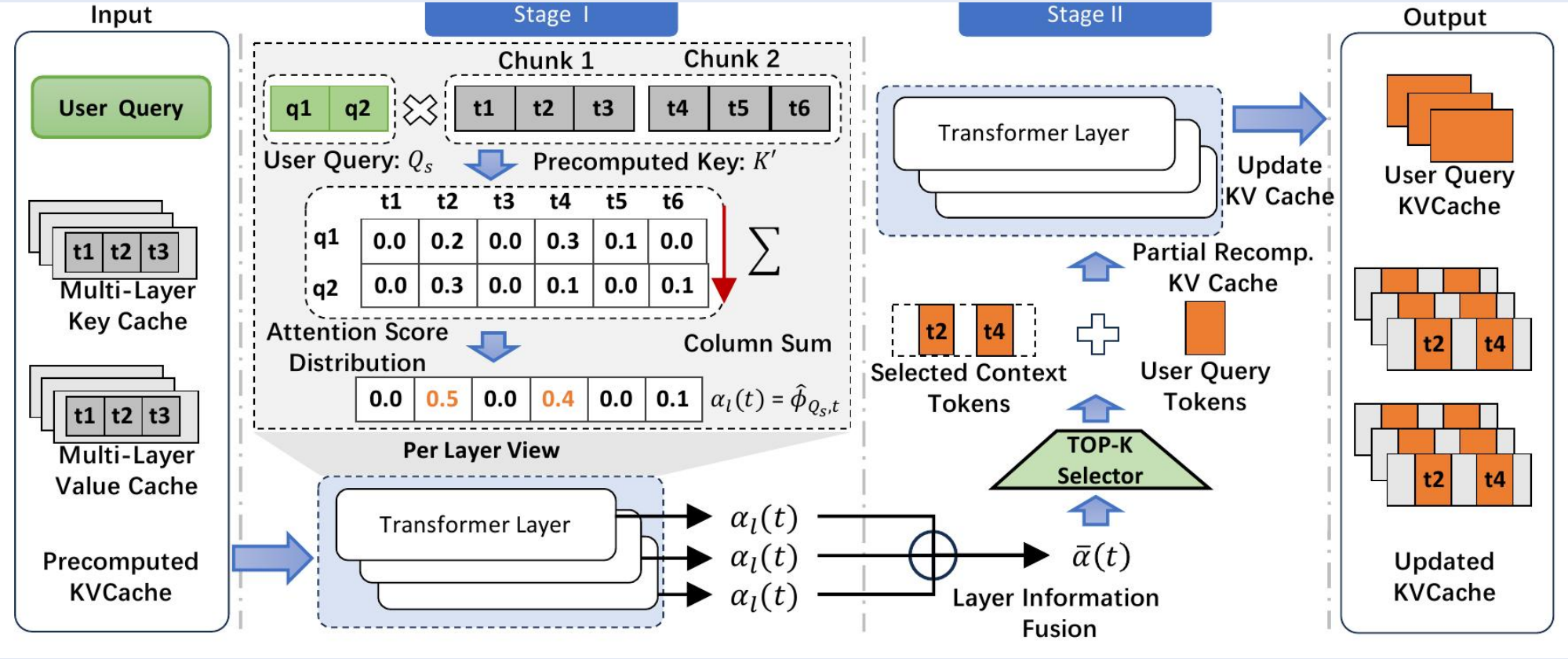
选择所用的层	RULER-MV	LB-HQA	LB-MQue
第一层	65.5	47.8	21.4
1/4 深度	92.0	48.0	20.5
1/2 深度	99.3	52.2	26.9
3/4 深度	98.0	51.8	23.3
最后一层	94.0	50.9	24.4
全层平均	100.0	50.7	24.4
全量 prefill	100.0	53.4	26.2

Llama-3.1-8B；LB = LongBench

只看第一层差得远；中间层与全层平均都接近全量。最优的单层因模型和任务而异，全层平均更稳，不需要逐模型调参

来源：Wang et al., ProphetKV, ICML 2026 poster, Fig. 7；表格数据来自作者在 OpenReview 审稿回复中补充的分层消融

ProphetKV：先发现，再重建



Stage I 发现：问题在复用 KV 上逐层前向，得到每层打分

Stage II 重建：对选中的 token 全层重算，更新 KV

来源：Wang et al., ProphetKV, ICML 2026 poster, Fig. 5

两个阶段怎么执行：先出名单，再按固定形状连续算

Stage I：发现

1. 输入：拼接好的复用 K / V + 问题 token
2. 每层：只算问题对上下文的注意力
3. 按列汇总，得到该层每个 token 的分数 $\alpha_l(t)$
4. 跨层平均：$\bar{\alpha}(t) = (1/L)\ \Sigma\ \alpha_l(t)$，取 Top-p

逐层边选边算

第 1 层选 → 第 2 层等 → 第 2 层选 → ……

Stage II：重建

1. 名单固定：所有层用同一组 token
2. 逐层重算选中 token 的 K / V
3. 隐藏状态沿层连续传递，不用停下来等决策
4. 写回 KV 后，正常开始解码

两阶段

先拿到全局名单 → 形状固定，连续执行

和推理引擎的对接点



蓝色为引擎需要新增或改造的环节

块级 KV 查找与拼接

按文档块内容哈希查找，而不是前缀哈希；Key 以不带 RoPE 的形式存储，拼接后按新位置重新施加

部分 token 的 prefill

只对选中 token 计算新的 K / V，同时读取其余 token 的复用 KV 参与注意力

拿到注意力权重

Stage I 需要显式的注意力分数；FlashAttention 默认不输出，需要专门的 kernel

现状：vLLM / SGLang 都没有「复用一部分、重算一部分」的公开接入点；LMCache 的 CacheBlend 在新架构下的接入仍在开发中（截至 2026 年 10 月）

05

实验结果

精度、重算比例与 TTFT

实验设置：模型、数据集、基线与硬件

模型	Llama-3.1-8B-Instruct、Qwen2.5-14B-Instruct、Qwen3-14B（推理模式）
数据	RULER（8K 上下文）、LongBench；文档按 512 token 切块
基线	全量重算、直接拼接、EPIC、CacheBlend、KVShare
预算	主结果为 20% 重算比例；另测 1%–100% 的曲线
时延	H100，TTFT；上下文 4K / 8K / 16K

需要注意

基于 HuggingFace Transformers 实现，未使用 FlashAttention 等融合算子；上下文最长 16K。时延的绝对值不代表生产引擎，相对比例更有参考意义

来源：Wang et al., ProphetKV, ICML 2026 poster, Sec. 5

20% 预算下的精度：最接近全量

RULER 平均准确率（%）

模型	全量	直接拼接	EPIC	CacheBlend	KVShare	ProphetKV
Llama-3.1-8B	88.82	61.36	70.32	77.27	73.47	84.71
Qwen2.5-14B	90.28	62.83	74.04	78.11	73.35	88.60
Qwen3-14B	91.79	43.85	67.94	71.99	70.64	89.89

LongBench 平均（%）

模型	全量	CacheBlend	ProphetKV
Llama-3.1-8B	50.19	39.82	50.80
Qwen2.5-14B	54.86	35.40	53.43
Qwen3-14B	61.06	50.17	59.52

⩾95%

精度保留，基本与全量持平；个别任务略超全量，属评测噪声范围

+8.8% ~ +24.9%

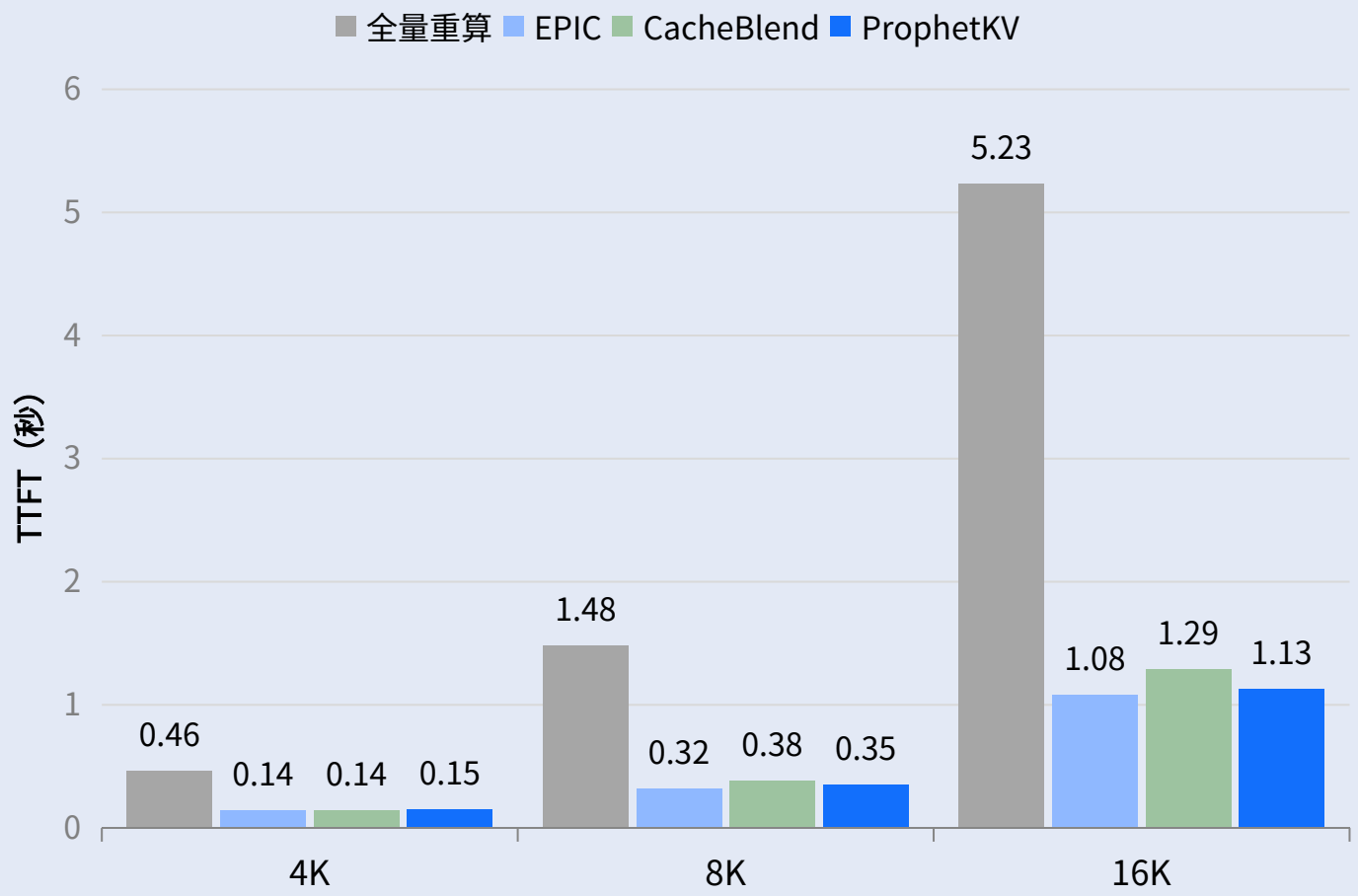
RULER 上相对最强基线的提升

+18.6% ~ +50.9%

LongBench 上相对最强基线的提升

来源：Wang et al., ProphetKV, ICML 2026 poster, Tab. 1；更多曲线见附录

TTFT：16K 时从 5.23 s 降到 1.13 s



4.6×

对比全量重算；Llama-3.1-8B / H100 / 16K

16K 时延拆分

Stage I 0.05 s + Stage II 1.08 s

不含从外部存储读回 KV 的时间；与 EPIC、CacheBlend 时延同一量级

来源：Wang et al., ProphetKV, ICML 2026 poster, Tab. 2；时延拆分来自 OpenReview 作者回复

06

存储视角：跨块复用之后的成本账

I/O 换算力，什么时候划算

复用单位降到块：共享存储从「可选」变成「必需」

120 GB/s	8 节点	PB 级	↓70%	↓67%
当前实测 · 受 PCIe 配置限制，固件优化中	单台并发支持	单节点最大容量	功耗	TCO

炎融 YRCache ContextBOX



4×BlueField-3 DPU · RDMA/RoCE · 26×U.2 NVMe

G1 GPU 显存	单卡 · 最快、最小
G2 主机内存	单节点 · 暂存
G3 本地 SSD	单节点 · 只能本机用
G3.5 ContextBox	**跨节点共享 · 多实例共用；经 YRCache / LMCache / Mooncake 接入**
G4 共享文件 / 对象存储	持久化 · 冷数据，不在关键路径

越往上越快、越小；越往下容量越大、共享范围越广 · 分层叫法参考 NVIDIA CMX

复用单位：会话前缀 → 文档块 → 可缓存对象：整个知识库 → **多实例共享同一份块 KV**

I/O 换算力的盈亏点

划算的条件

读回 KV + 问题打分 + 部分重算 < 全量 prefill

盈亏带宽 ≈ KV 大小 ÷ 节省的时间

节省的时间 = T全量 − TProphetKV

读带宽高于它，复用才省时间

上下文	KV 大小	省下的时间	盈亏带宽
4K	0.54 GB	0.31 s	≈ 1.7 GB/s
8K	1.07 GB	1.13 s	≈ 0.95 GB/s
16K	2.15 GB	4.10 s	≈ 0.52 GB/s

估算：Llama-3.1-8B，KV 128 KiB/token（32 层 × 8 个 KV 头 × 128 维 × K、V × FP16）；时间取论文 HuggingFace 实测

上下文越短，越不划算

prefill 是平方增长、读 KV 是线性增长；短上下文的盈亏带宽更高

引擎越快，门槛越高

生产引擎 prefill 快数倍，省下的时间缩短，所需带宽同比上升

命中率决定上限

未命中的块仍要完整 prefill；被重算 token 的 KV 只用于打分，之后被覆盖

两阶段的一短一长：加载难重叠，降级有时机

CacheBlend：加载与重算逐层重叠



TTFT ≈
max(加载, Stage I) + Stage II

示意图，非比例

ProphetKV：Stage II 要等所有层就绪（短板）



反过来，这也是好处：Stage II 开始前有一个决策点，超时降级就放在这里

每块设截止时间

按块大小和当前带宽估算加载时间；超时即视为未命中

超时块整体重算

该块全部 token 并入 Stage II 名单，相当于对这个块做 prefill；Stage II 尚未开始，不需要回退

打分不等待

Stage I 先在已到达的块上打分；超时块不参与排序，直接进入重算名单

设计分析，非论文内容；需在 YRCache 中验证

后续进展：按问题选择正在成为共识

QCFuse

浙江大学、蚂蚁集团等 · 2026

- 问题只与每块的少量锚点 token 计算注意力
- 离线选出 3 个关键中间层，不再遍历所有层
- 选择与 KV 加载流水，补上 ProphetKV 的系统短板
- 同等质量下 TTFT 比 ProphetKV 快约 1.5 倍

开源研究原型：基于 SGLang 0.5.4 修改，未并入 SGLang 主线

RelaxKV

上海交通大学、中国电信 TeleAI · 2026-09

- 沿用问题注意力打分
- 重算时只关注与问题相关的上下文，而不是全部前文
- LongBench 平均分（4 个模型）、LV-Eval 上精度高于 ProphetKV
- RULER-MV 32K（Qwen3-14B）：95.0，ProphetKV 91.2，全量 95.8

论文实现，未见引擎集成；重算口径不同（15% 锚点 vs 20% 重算），计算量不能直接对比

共识：按问题选 token；下一步是与加载流水结合、缩小重算的上下文。两者都把 ProphetKV 作为主要基线

来源：Yan et al., QCFuse, arXiv 2606.05875；Qi et al., RelaxKV, arXiv 2609.33503

能力边界：编码场景下，问题未必能预示生成

为什么编码更难	应对方向（尚未验证）
相关内容不是「小子集」 「帮我重构这个函数」要整个函数、调用方和类型定义；「这里报错了怎么改」要报错栈加跨文件代码	**按打分分布自适应调预算** 打分很平 = 没有明显的相关子集 → 提高重算比例，必要时退回全量 prefill
关联常是间接的 被调用函数的定义与问题字面上并不相似，问题注意力可能看不到	**按代码结构切块，依赖一起钉住** 按函数 / 类切块；被编辑的函数、报错栈，连同被调函数和类型定义整块重算
生成很长 重构要输出大段代码，「问题预示生成关注点」会随生成漂移	**生成中途重新打分** 每生成一段，用新生成的 token 再打一次分，补算新关注到的块

定位：适合「上下文长、问题短、答案只用局部」的场景。同一会话内的追加，前缀缓存就够了；跨会话、跨 agent 以不同顺序重读同一批文件，才需要非前缀复用，这部分尚未验证

三个代价，一句结论

01

精度

20% 预算下仍有约 4% 相对损失；摘要、全文翻译等没有「小相关子集」的任务，以及问题不在末尾的模板，会明显退化

02

I/O

候选块的 KV 要全部读回显存；上下文短、存储慢、命中率低时，净收益会被压小

03

引擎

需要块级复用、部分重算、注意力权重输出等接口；主流引擎尚无公开接入点

先选对，再少算

跨块复用的瓶颈不在「能不能复用」，而在有限的重算预算给谁；用户问题的注意力给出了这个排序

20% 重算 · 精度基本与全量持平 · prefill 加速 4.6×

QCon
全球软件开发大会

InfoQ 技术大会

谢谢

ProphetKV：由用户问题驱动的选择性重算

彭德跃 · 北京焱融科技 首席架构师

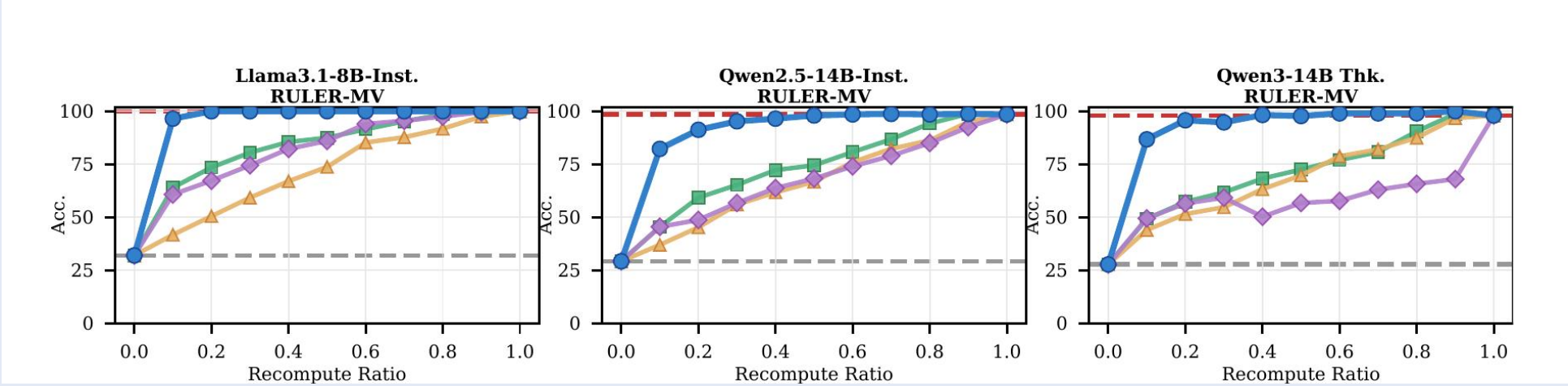
Q&A



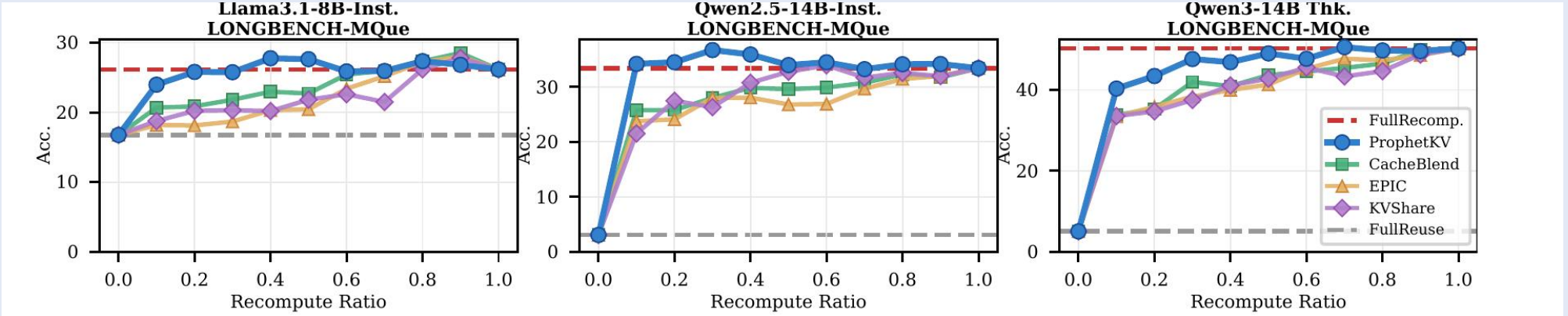
附录

备用页

附录 A：精度随重算比例的变化



RULER MultiValue



LongBench Musique；红色虚线为全量重算，灰色虚线为直接拼接

来源：Wang et al., ProphetKV, ICML 2026 poster, Fig. 8

附录 B1：选择性重算路线

时间	工作	选择 / 修复方式	与 ProphetKV 的关系
2023-11	Prompt Cache（MLSys'24）	模块化复用，不重算	非前缀复用的起点
2024-05	CacheBlend（EuroSys'25）	KV 偏差，逐层收缩；与加载流水	选择性重算的起点
2024-10	EPIC（ICML'25）	每块开头少量 token	静态选择
2025-02	Cache-Craft（SIGMOD'25）	识别上下文敏感 token；缓存多个变体	按块管理，可跨请求复用
2025-03	KVShare	隐状态偏差	动态选择，与问题无关
2025-08	SamKV（AAAI'26）	多上下文稀疏化后局部重算	可与按问题选择叠加
2025-10	CacheClip	辅助小模型的查询注意力	也用查询信号，需辅助模型
2025-11	A^3	第一层的问题注意力	思路最接近，只用单层
2026-01	FusionRAG	离线预融合 + 在线问题引导	可互补
2026-02	ProphetKV（ICML'26 poster）	全层平均的问题注意力	本次分享
2026-06	QCFuse	锚点 + 关键层，与加载流水	补系统效率
2026-09	RelaxKV	问题打分 + 只看相关上下文重算	补精度

附录 B2：其它路线与评测

类别	工作	做法 / 结论
训练 / 注意力重塑	Block-Attention（ICLR'25）、TurboRAG	微调模型适应块内独立注意力，推理时不重算
训练 / 注意力重塑	APE（ICLR'25）	并行编码 + 注意力校准，免训练
训练 / 注意力重塑	KVLink、KV Packet	可训练的连接 token / 首尾 soft token，免重算
训练 / 注意力重塑	Attuner（2026-09）	问题一侧加 LoRA，蒸馏全量输出，免重算
注意力解耦	DAF（2026-07）	重要 token 跨文档注意力 + 问题-文档注意力，再融合
系统	MEPIC（2025-12）	把位置无关缓存做进 vLLM 分页 KV，HBM 内跨请求共享
评测	分块缓存复用实验研究（2026-03）	单一方法达不到全量；组合方案可再提升约 5%
评测	复用精度评测（2026-09）	LongBench 口径会高估复用效果；提出 stale cache 问题
评测	KVShareArena（2026-09）	重算类在模型版本变化时比训练类更稳
安全	HijackKV（2026-07）	共享的块 KV 可被对抗前缀「投毒」

附录 C：理论上的近似与适用边界

用问题代替生成 token	经验假设；长推理、长生成时可能漂移
三角不等式上界	忽略了误差之间的相互抵消，按上界选出的集合不一定最优
重算即恢复真值	被重算 token 仍会 attend 到未重算的 token，只能部分恢复
用 Φ' 代替 $\lvert\Delta\Phi\rvert$	Φ' 在近似 KV 上计算；「复用后才不被关注」的 token 可能被漏选

附录 D：问题放在前面时会怎样

模型	提示顺序	LB-MQue	LB-HQA	LB-WQA	RULER-VT	RULER-QA_1
Llama-3.1-8B	问题在前	15.8	35.3	27.2	62.0	28.33
Llama-3.1-8B	问题在后	28.6	54.0	44.0	82.2	72.8
Qwen2.5-14B	问题在前	34.7	60.2	52.5	100.0	39.1
Qwen2.5-14B	问题在后	34.5	64.3	56.9	94.6	71.2
Qwen3-14B	问题在前	35.0	58.5	63.7	100.0	58.08
Qwen3-14B	问题在后	34.9	59.5	66.8	99.6	73.08

读法：问题在前时，因果掩码下问题看不到后面的上下文，先验变弱；多数任务明显下降，少数任务持平。工程上可在末尾追加或复述问题

数据：ProphetKV 作者在 OpenReview 审稿回复中补充的实验（ICML 2026 poster）

AiCon 北京站

全球人工智能开发与应用大会

探索 AI 应用边界

Explore the limits of AI applications

2026 年 12 月 18–19 日 / 北京新云南皇冠假日酒店

参会咨询

专题：Post-Training Engineering | 数据、奖励与模型能力塑造



专题出品人：余瑶

飞猪 / 资深算法专家

专题：大模型推理与算力调度实践



专题出品人：范苑

阿里云 / 专有云计算产品负责人

专题：Knowledge Engineering | 让 Agent 理解企业



专题出品人：盛茂家

火山引擎 / Viking 团队技术负责人

专题：Loop & Graph Engineering



专题出品人：孙子荀

腾讯 / 腾讯高级总监

专题：AI 原生架构的演进与治理



专题出品人：刘中兵

快手 / 架构治理团队 & AI 架构演进负责人

专题：Agentic Software Engineering | 从代码生成到软件交付



专题出品人：杨杰

荣耀 / 首席 IT 技术架构师